

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Д34

Выполнил:

Саид Наваф

Группа

К3341

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Задание:

1. Разделите ваше монолитное бэкэнд-приложение на микросервисы, придерживаясь концепции database-per-service
2. Спроектируйте микросервисную архитектуру: отразите взаимосвязь между микросервисами, опишите способы их взаимодействия друг с другом
3. Спроектируйте разделение вашей БД на обособленные БД
4. Спроектируйте и опишите эндпоинты, необходимые для обеспечения межсервисного взаимодействия в формате спецификации OpenAPI
5. Опишите варианты запросов и ответов, задокументируйте возможные ошибки
6. В результате вы должны сформировать документ, в котором будут описаны конкретные требования и шаги для достижения разделения вашего монолитного бэкэнд-приложения на отдельные микросервисы

Ход работы

1. Анализ текущей архитектуры:
Выявлены ограничения монолита (единая кодовая база на Go, общая БД PostgreSQL): сложность независимого масштабирования, единая точка отказа, блокирующие миграции схемы БД.
2. Проектирование микросервисов:
Приложение разделено на 4 независимых сервиса по бизнес-доменам:

`Auth Service` — регистрация, логин, JWT-токены, управление ролями.

`Job Service` — CRUD вакансий, категории.

`Candidate Service` — управление резюме соискателей.

`Interaction Service` — отклики на вакансии, смена статусов.

3. Разделение баз данных (`database-per-service`):
Каждая сущность вынесена в изолированную БД: `auth_db`, `job_db`, `cand_db`, `interaction_db`. Прямые `FOREIGN KEY` между базами удалены. Целостность данных поддерживается на уровне приложения через валидацию и eventual consistency.
4. Схема взаимодействия сервисов:
Спроектирована маршрутизация через API Gateway. Синхронное взаимодействие (REST) используется для валидации данных перед

операциями, асинхронное (Event-Driven через RabbitMQ/Kafka) — для каскадных обновлений.

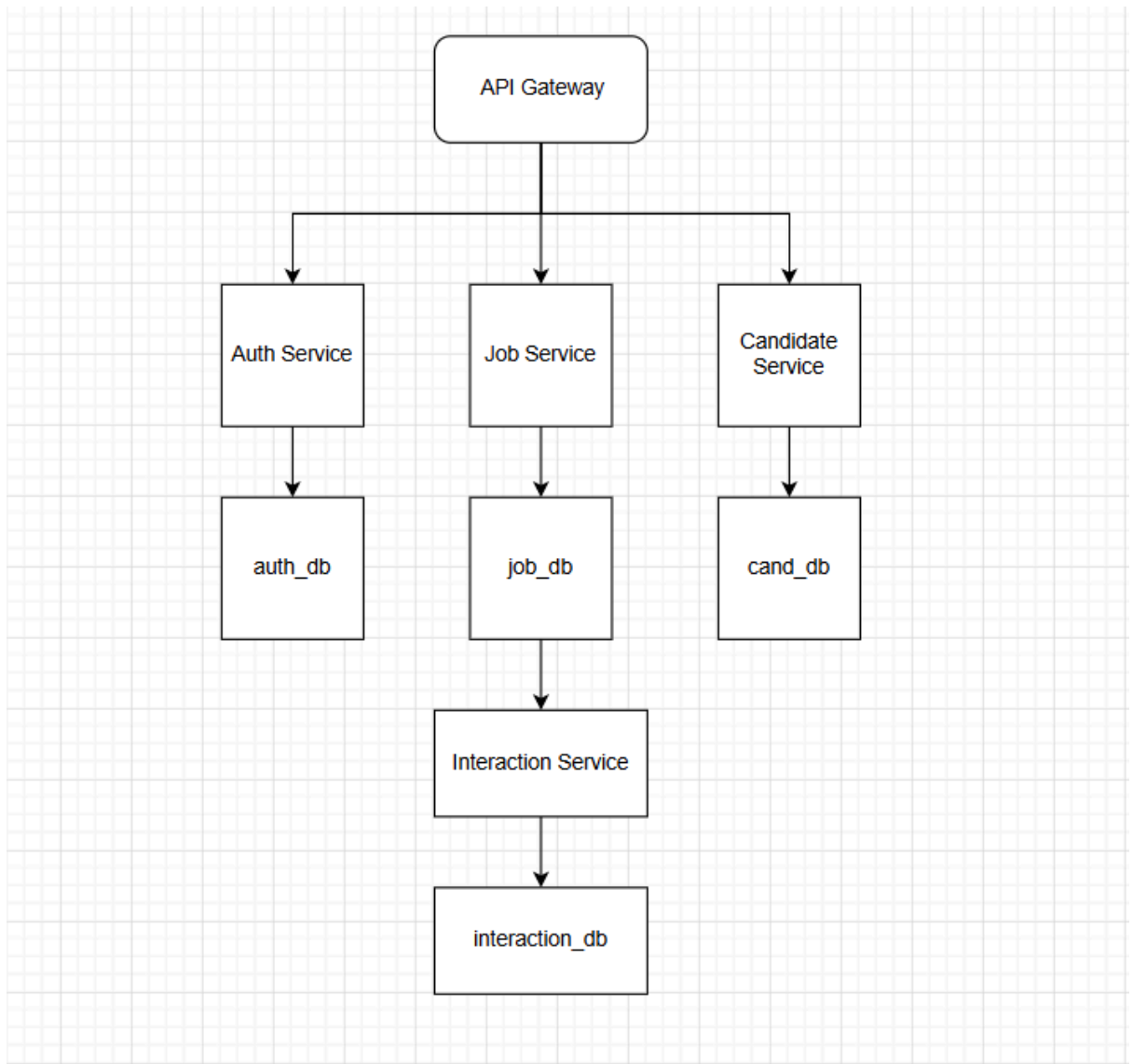


Рис. 1. Схема микросервисной архитектуры Job Search API

5. Спецификация OpenAPI:

Разработан файл [openapi.yaml](#), описывающий внутренние контракты взаимодействия. Примеры эндпоинтов:

[GET /service/users/{id}](#) → проверка существования пользователя (Auth Service)

[GET /service/vacancies/{id}](#) → проверка активности вакансии (Job Service)

[GET /service/resumes/{id}](#) → проверка владения резюме (Candidate Service)

Описаны схемы ответов, обязательные поля и коды ошибок ([400](#), [404](#), [500](#), [503](#)).

6. План миграции:

Определён поэтапный переход без остановки продакшена: выделение
Auth → Job → Candidate → Interaction → внедрение брокера сообщений
→ отключение монолита.

Вывод

В ходе работы успешно спроектирована микросервисная архитектура для системы поиска работы. Монолит разделён на 4 изолированных сервиса с собственными базами данных, что обеспечивает независимое масштабирование, отказоустойчивость и гибкость развёртывания. Разработана чёткая спецификация OpenAPI для межсервисного взаимодействия, описаны стратегии поддержания целостности данных и обработки распределённых ошибок. Архитектурный дизайн готов к поэтапной реализации и соответствует современным стандартам backend-разработки.